

QuestTable



Università degli Studi di Roma Tor Vergata

Matteo Marcoccia

1. SPECIFICA DEI REQUISITI SOFTWARE

1.1 Introduzione

1.1.1 Scopo del Documento

Il presente documento ha l'obiettivo di definire in modo dettagliato le specifiche dei requisiti, l'architettura logica e le scelte progettuali adottate per lo sviluppo del sistema "QuestTable".

Lo scopo principale è fornire una guida esaustiva che illustri:

- I **requisiti tecnici**, sia hardware che software, necessari per il corretto funzionamento dell'applicazione.
- Le **funzionalità** offerte dal sistema attraverso l'analisi delle User Stories e dei Casi d'Uso.
- La **struttura del design**, documentata mediante diagrammi UML (VOPC e Design-Level), per facilitare la comprensione dell'interazione tra le classi e l'applicazione dei design pattern.
- Le metodologie di **validazione e testing** utilizzate per garantire la robustezza del codice e la conformità ai requisiti espressi.

Questo documento è destinato sia agli sviluppatori, come riferimento per l'implementazione e la manutenzione futura, sia agli stakeholder, per verificare che tutte le necessità utente siano state correttamente interpretate e soddisfatte nel ciclo di vita del software.

Redatto da: **Matteo Marcoccia**.

1.1.2 Panoramica del Sistema

Il sistema **QuestTable** è una piattaforma software sviluppata in linguaggio **Java**, progettata per digitalizzare e ottimizzare l'esperienza d'uso all'interno di un "Board Game Cafe". L'applicazione è strutturata seguendo l'architettura **MVP**, che permette una separazione netta tra la logica di business e la presentazione dei dati.

Il sistema offre una soluzione integrata che unisce la gestione delle attività in loco alla componente di vendita, includendo le seguenti funzionalità principali:

- **Prenotazione Posti ai Tavoli:** Gli utenti possono esplorare il catalogo dei giochi da tavolo disponibili nel locale e prenotare un posto a sedere per una sessione di gioco specifica. Questo permette al locale di gestire i flussi di clienti e di facilitare

l'aggregazione di giocatori per partite pubbliche.

- **E-commerce e Acquisto Giochi:** QuestTable integra un modulo per l'acquisto diretto dei giochi da tavolo. Gli utenti possono consultare i prezzi, la disponibilità in magazzino e procedere all'ordine dei propri titoli preferiti.
- **Sistema di Fidelizzazione (Loyalty Points):** Per incentivare la partecipazione, il sistema include una logica di punti fedeltà. Gli utenti accumulano punti effettuando prenotazioni e partecipando alle attività del locale; tali punti possono poi essere riscattati come sconti durante l'acquisto di giochi fisici.
- **Persistenza dei Dati tramite MySQL:** La gestione delle informazioni (profili utente, storico prenotazioni, catalogo prodotti e saldo punti) è affidata a un database relazionale **MySQL**. Questa scelta garantisce l'integrità dei dati, la sicurezza e la possibilità di gestire query complesse per i report di vendita e prenotazione.
- **Interfaccia Utente Moderna:** La GUI è realizzata tramite **JavaFX**, offrendo un'interfaccia desktop reattiva, intuitiva e arricchita da elementi grafici che migliorano l'esperienza dell'utente finale.

Il sistema gestisce diverse tipologie di utenti, ognuna con permessi e funzionalità specifiche: **Cliente** e **Gestore del Locale**.

1.1.3 Requisiti Hardware e Software

Per garantire il corretto funzionamento e lo sviluppo del sistema **QuestTable**, sono stati identificati i seguenti requisiti tecnici:

Requisiti Software:

- **Sistema Operativo:** Microsoft Windows 10/11, macOS o distribuzioni Linux recenti.
- **Linguaggio di Programmazione:** Java (JDK 17 o versioni successive).
- **Ambiente di Sviluppo (IDE):** IntelliJ IDEA o Eclipse con supporto per il plugin Maven.
- **Librerie Grafiche:** JavaFX SDK per la gestione dell'interfaccia utente.
- **Database Management System:** MySQL Server (versione 8.0+) per la persistenza dei dati e MySQL Workbench per la gestione del database.
- **Build Tool:** Apache Maven per la gestione delle dipendenze e l'automazione del build.
- **Version Control:** Git, con repository ospitato su GitHub.
- **Analisi del Codice:** SonarCloud per l'analisi statica della qualità del codice.

Hardware requirements:

- **Processore:** Architettura x64 o ARM (es. Apple Silicon) con frequenza minima di 2.0 GHz.
- **Memoria RAM:** Minimo 4 GB (consigliati 8 GB o superiore per l'esecuzione contemporanea dell'IDE e del server MySQL).
- **Spazio su Disco:** Almeno 500 MB di spazio libero per l'installazione delle dipendenze e la memorizzazione del database locale.
- **Connessione Internet:** Necessaria per la sincronizzazione del repository, il download delle librerie tramite Maven e l'eventuale connessione al database remoto.

1.1.4 Analisi di piattaforme simili

Per lo sviluppo di **QuestTable**, sono state analizzate alcune piattaforme esistenti che offrono servizi di prenotazione o e-commerce per il mondo del gaming, al fine di identificare best practice e aree di miglioramento.

Tabletop Simulator / Board Game Arena:

- *Pro:* Offrono un'esperienza di gioco digitale completa e immediata.
- *Contro:* Mancano totalmente dell'integrazione con un luogo fisico e non gestiscono la vendita di prodotti reali o la prenotazione di spazi fisici.

Eventbrite / Meetup:

- *Pro:* Eccellenti per la gestione di eventi e prenotazioni di posti.
- *Contro:* Sono piattaforme generiche. Non offrono un catalogo dedicato ai giochi da tavolo né un sistema di fidelizzazione (punti fedeltà) integrato con l'acquisto di prodotti fisici nello stesso locale.

Siti E-commerce specializzati (es. DungeonDice):

- *Pro:* Gestione del catalogo e della vendita molto efficiente.
- *Contro:* Non hanno una componente legata alla socialità o alla prenotazione di tavoli in un locale fisico.

Vantaggio Competitivo di QuestTable: A differenza delle piattaforme analizzate, **QuestTable** propone una soluzione "all-in-one" che unisce il mondo digitale a quello fisico. Il valore aggiunto risiede nel **sistema di fidelizzazione circolare**: l'utente viene incentivato a frequentare il locale (prenotazione tavoli) ricevendo in cambio vantaggi tangibili sull'acquisto di prodotti (e-commerce), creando un ecosistema completo per l'appassionato di giochi da tavolo.

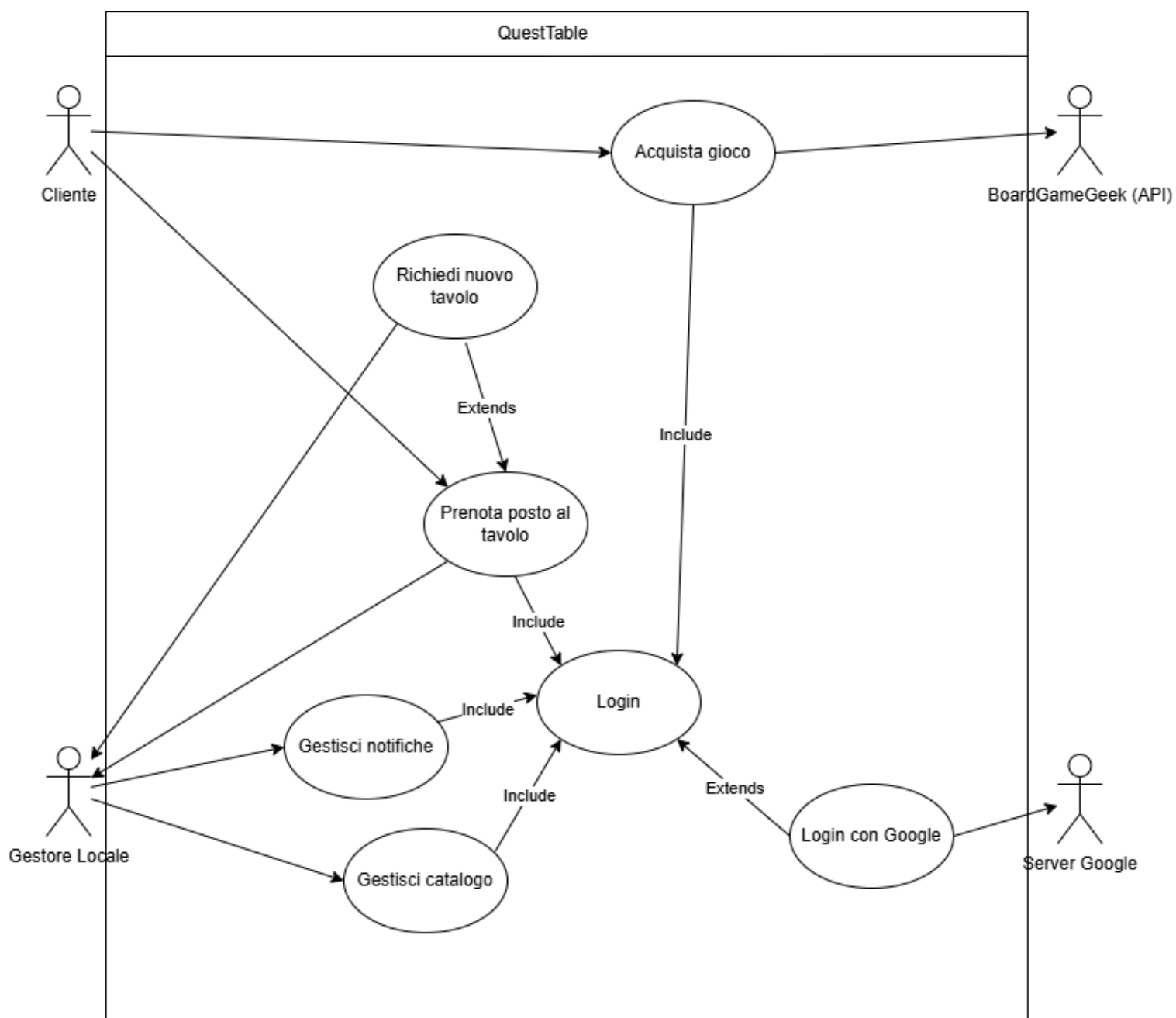
1.2 User Stories

- 1.2.1 Come cliente, voglio prenotare posti ad un tavolo da gioco, così da assicurarmi la partecipazione e accumulare punti fedeltà.
- 1.2.2 Come cliente, voglio richiedere l'apertura di un tavolo personalizzato, per garantire l'esperienza di gioco desiderata.
- 1.2.3 Come cliente, voglio utilizzare i punti fedeltà per acquistare giochi dal catalogo, così da ridurre il prezzo finale.

1.3 Requisiti funzionali

- 1.3.1 Il sistema deve registrare la prenotazione di posti ad un tavolo da gioco, e accreditare punti fedeltà al cliente.
- 1.3.2 Il sistema deve registrare la richiesta di nuovi tavoli su proposta del cliente.
- 1.3.3 Il sistema deve fornire al cliente l'insieme di giochi da tavolo acquistabili, accettando i punti fedeltà come credito scalabile dal prezzo di listino.

1.4 Use Cases Diagram



1.4.1 Internal Steps

Use Case: **Prenota posto al tavolo.**

1. Il cliente effettua il Login.
2. Il sistema carica i dati dell'utente e i tavoli disponibili.
3. Il sistema presenta i tavoli disponibili.
4. Il cliente seleziona un tavolo.
5. Il sistema illustra i dettagli del tavolo selezionato (data e ora, prezzo, numero posti disponibili).
6. Il cliente seleziona il numero di partecipanti che vuole aggiungere.
7. Il sistema calcola il preventivo.
8. Il sistema presenta il preventivo al cliente.
9. Il cliente procede al pagamento del preventivo.
10. Il sistema elabora il pagamento.
11. Il sistema aggiorna i posti disponibili del tavolo e invia al gestore una notifica relativa a una nuova prenotazione.

Extensions:

- 3a. Non ci sono tavoli per gioco richiesto: Il cliente richiede un nuovo tavolo.
- 7a. Tavolo non più disponibile: Il sistema mostra un messaggio di errore e invita il cliente a selezionare un altro tavolo.
- 10a. Transazione non andata a buon fine: Il sistema mostra un messaggio di errore e invita il cliente a ritentare la transazione.

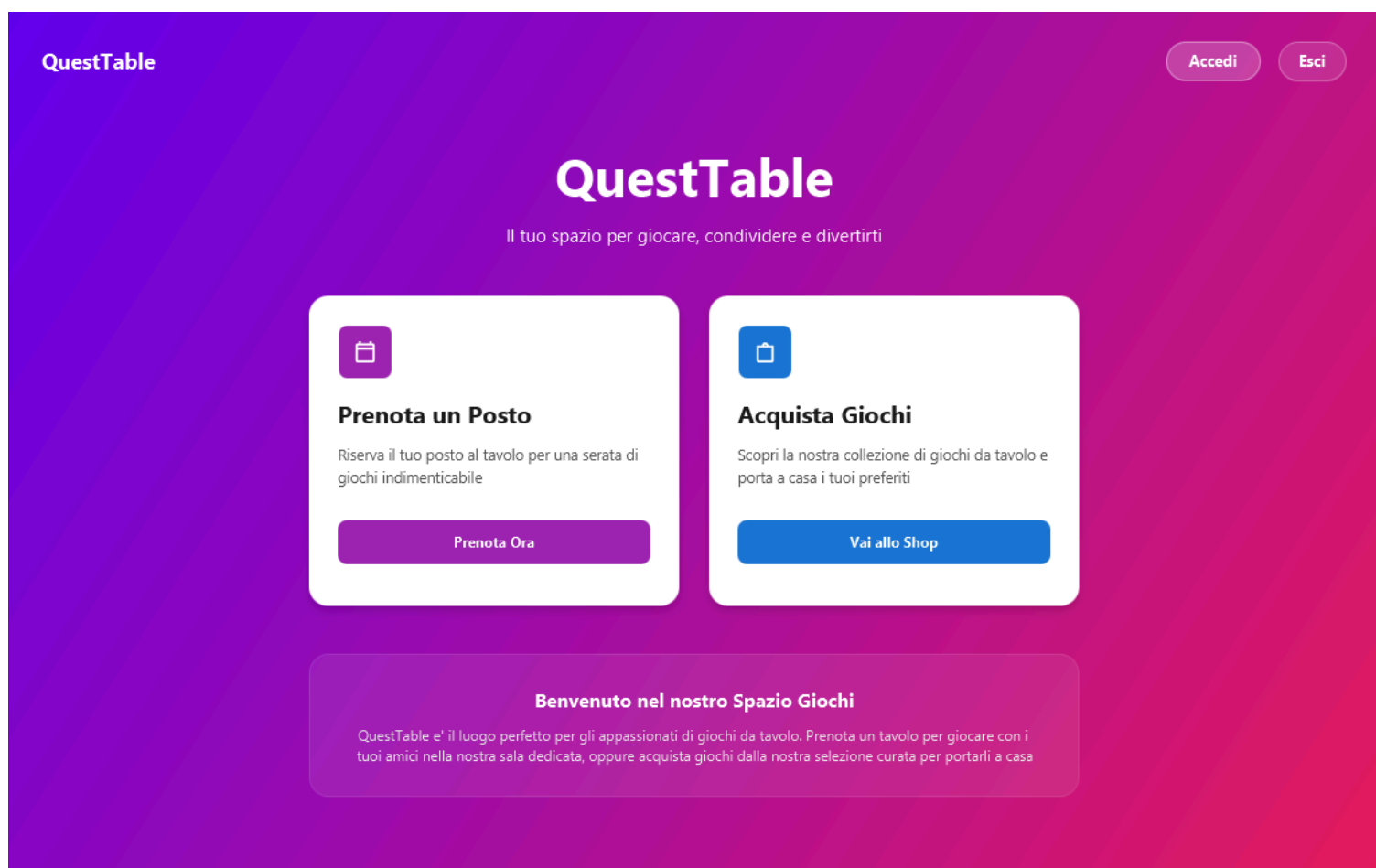
2. STORYBOARDS

In questa sezione viene presentato lo storyboard del sistema QuestTable, illustrando le principali interfacce grafiche attraverso le quali l'utente interagisce con i servizi di prenotazione e shop.

2.1 Pagina Iniziale

La pagina iniziale funge da punto di accesso principale, permettendo all'utente di scegliere tra la prenotazione di un tavolo o l'acquisto di giochi dal catalogo.

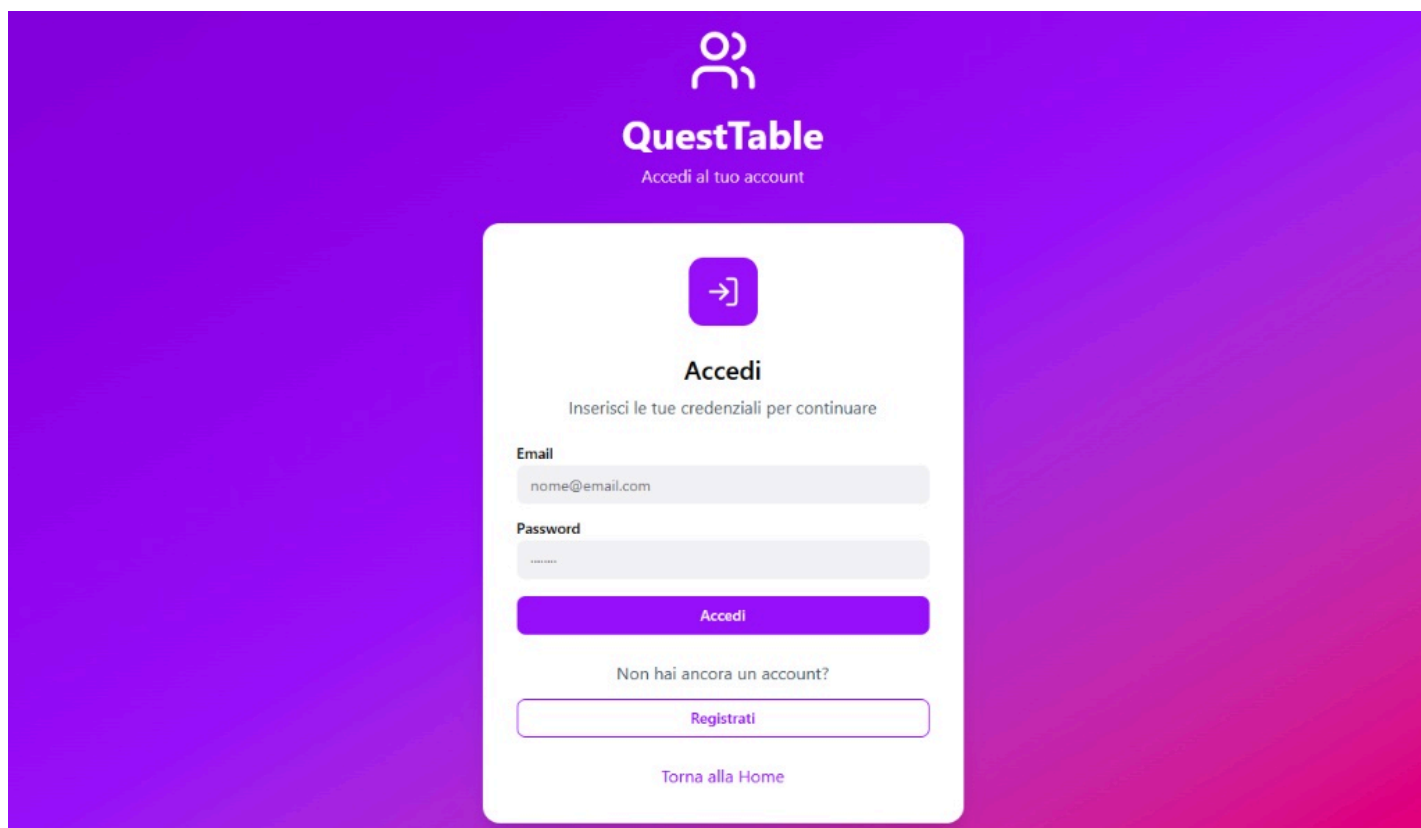
- **Elementi chiave:** Pulsanti rapidi "Prenota Ora" e "Vai allo Shop", e tasto di accesso in alto a destra.



2.2 Pagina di Login

L'interfaccia di autenticazione permette agli utenti di accedere al proprio profilo per gestire prenotazioni e utilizzare i punti fedeltà accumulati.

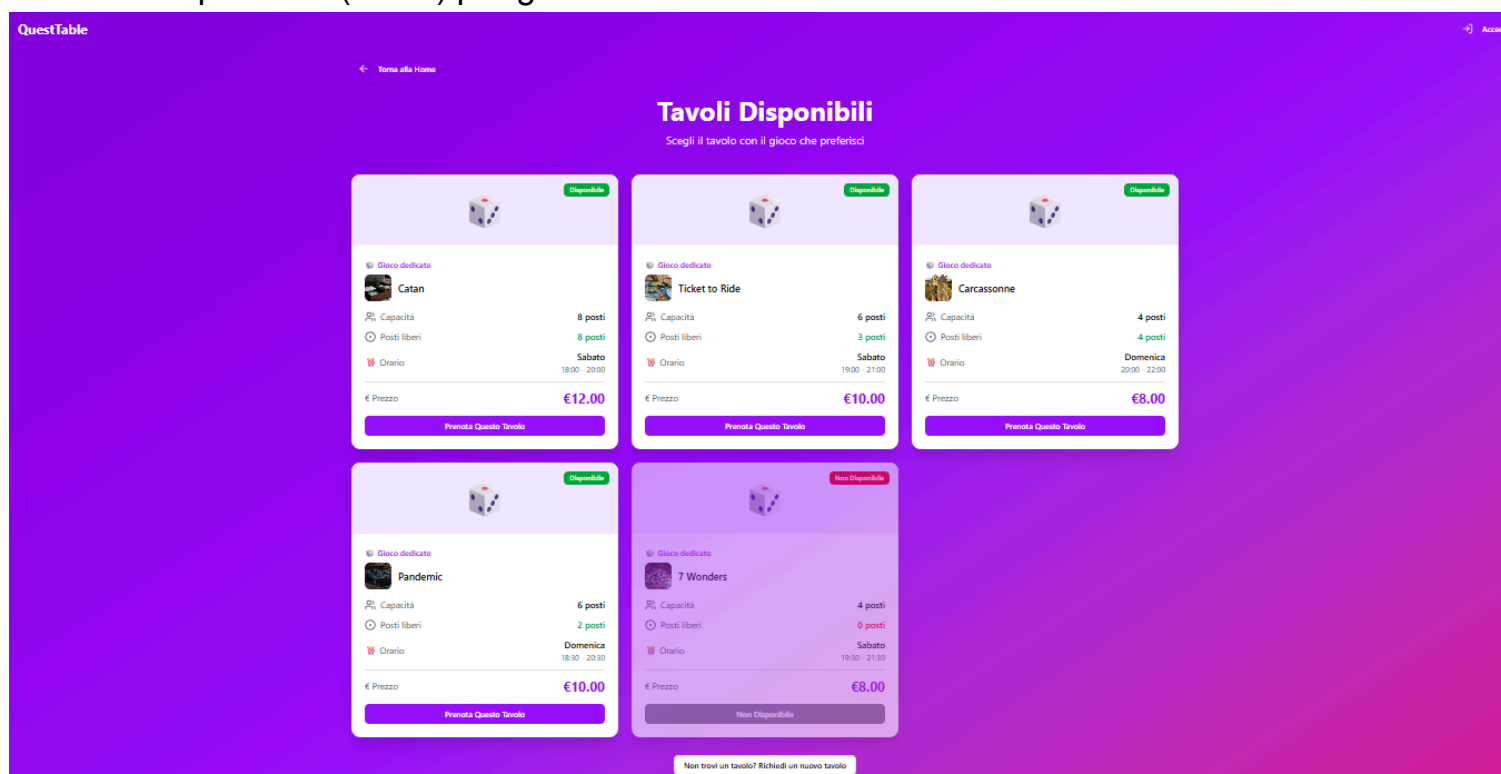
- **Elementi chiave:** Campi email e password, pulsante di login e link per la registrazione di un nuovo account.



2.3 Selezione Tavoli Disponibili

Questa schermata mostra la disponibilità in tempo reale della sala giochi, filtrata per gioco dedicato.

- **Feedback Visivo:** I tavoli sono contrassegnati da badge "Disponibile" (verde) o "Non Disponibile" (rosso) per guidare l'utente nella scelta.



2.4 Dettagli e Conferma Prenotazione

L'utente finalizza i dettagli della propria sessione di gioco prima della conferma definitiva.

- **Logica di Business:** Il sistema visualizza il riepilogo dei costi e il calcolo dei punti fedeltà che verranno accreditati con l'operazione.

QuestTable →] Accedi

← Torna ai Dettagli

Dettagli Prenotazione
Compila i dettagli per la tua prenotazione

Tavolo
Tavolo per Catan

Data e Orario

Data: 15/02/2026 Ora: 19:00

Numero Partecipanti
2

Riepilogo

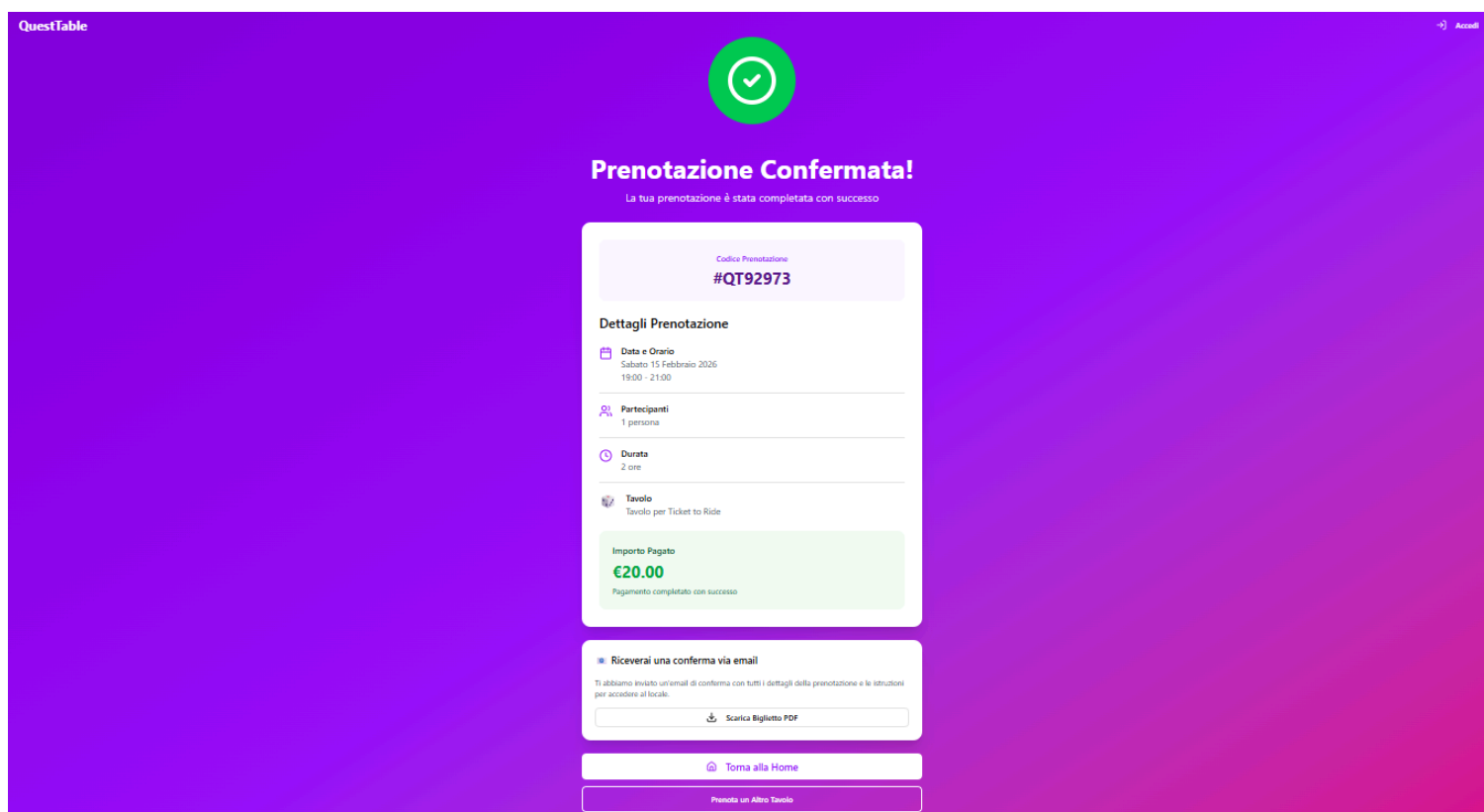
Tavolo	Catan
Partecipanti	2
Prezzo	€12.00
★ Punti Fedeltà	+240 punti
Costo stimato:	€24.00

Conferma la Prenotazione

2.5 Prenotazione Completata

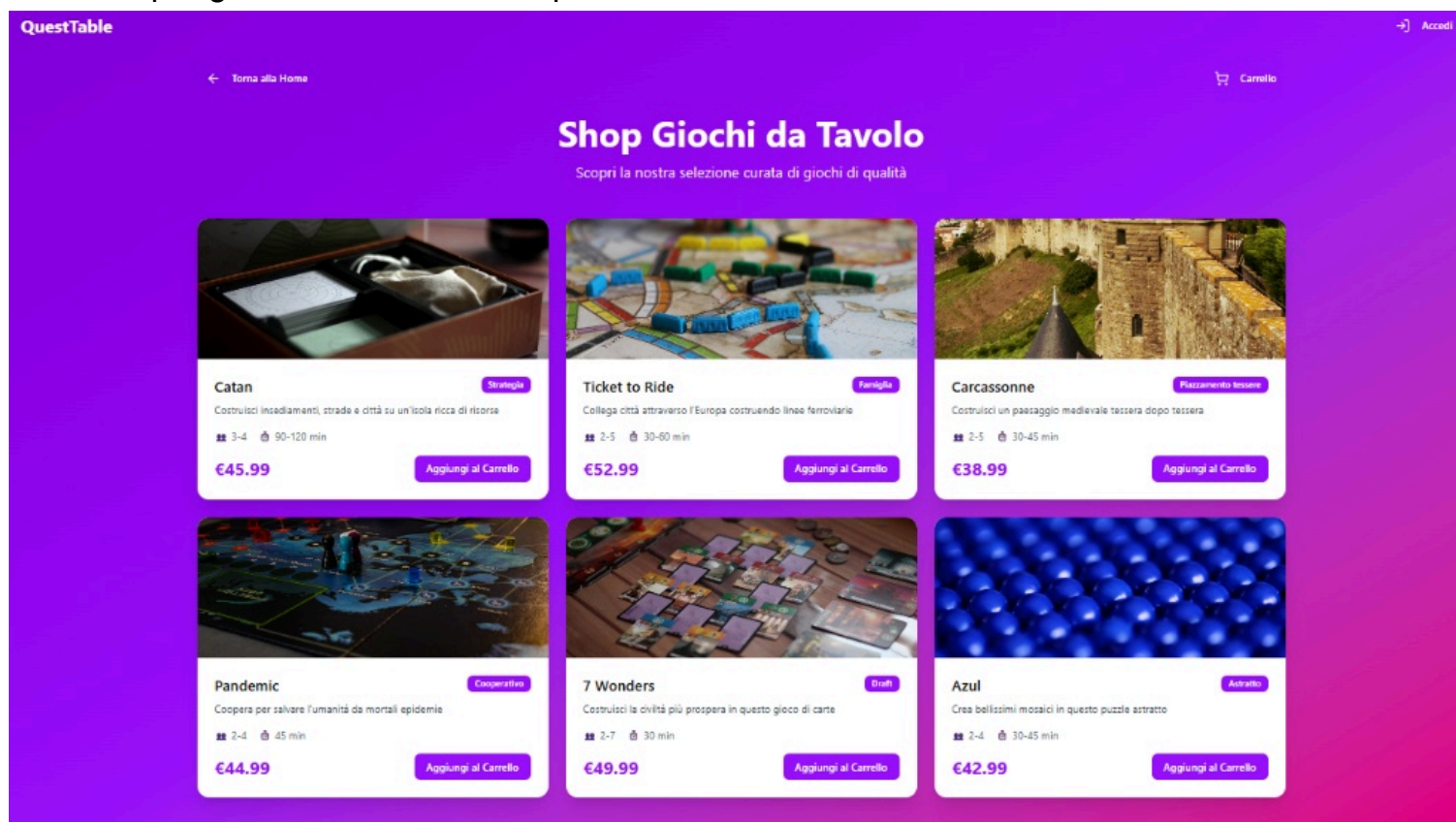
Schermata di successo che conferma l'avvenuta transazione e fornisce i dettagli per l'accesso al locale.

- **Elementi chiave:** Codice di prenotazione univoco, riepilogo importo pagato e opzione per scaricare il biglietto in formato PDF.



2.6 Shop Giochi da Tavolo

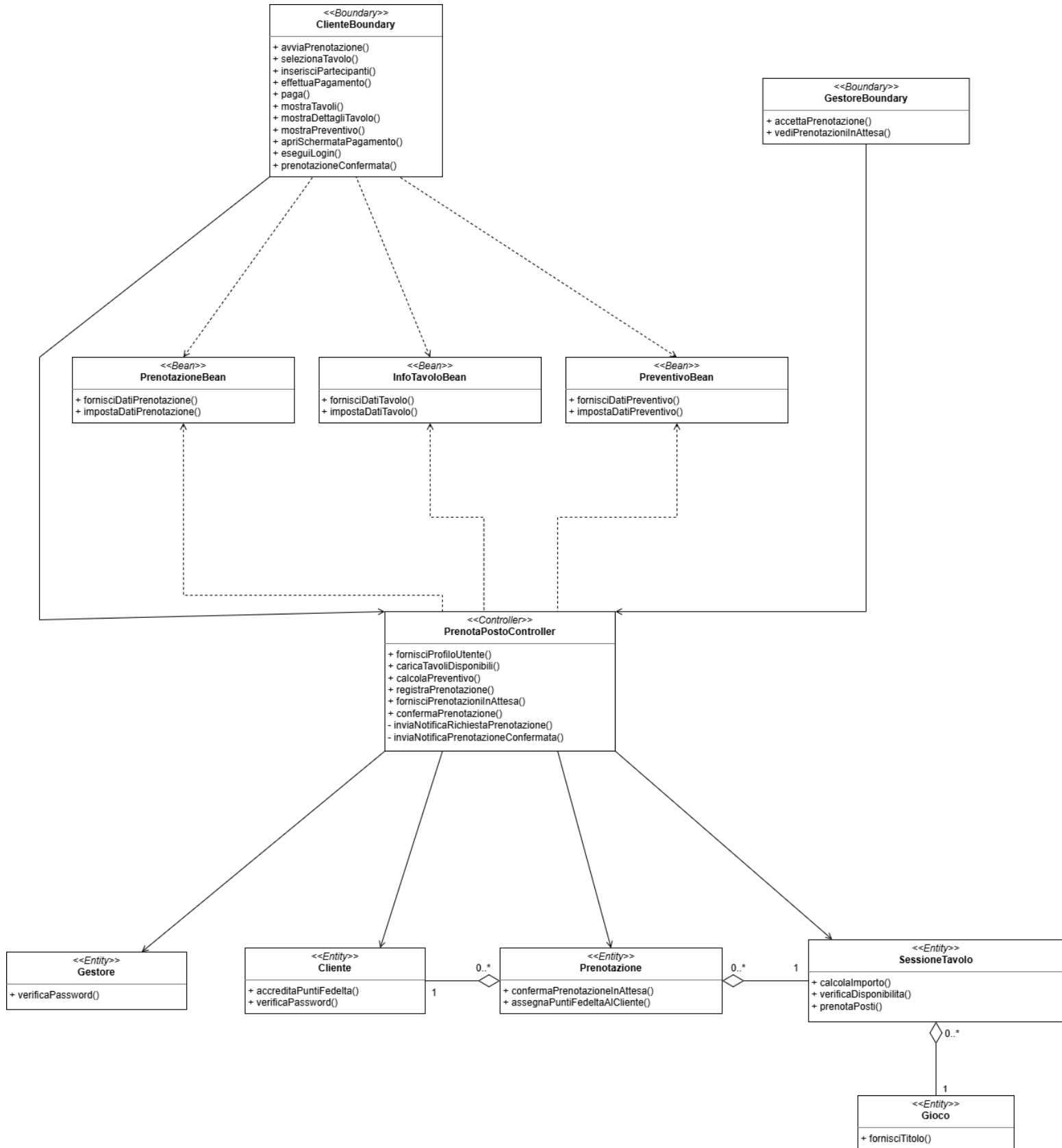
L'interfaccia dedicata all'e-commerce presenta una selezione curata di titoli acquistabili. Ogni articolo è catalogato per genere e mostra dettagli tecnici (durata, età, numero giocatori) per guidare l'utente nell'acquisto.



3. DESIGN

3.1 Class Diagram

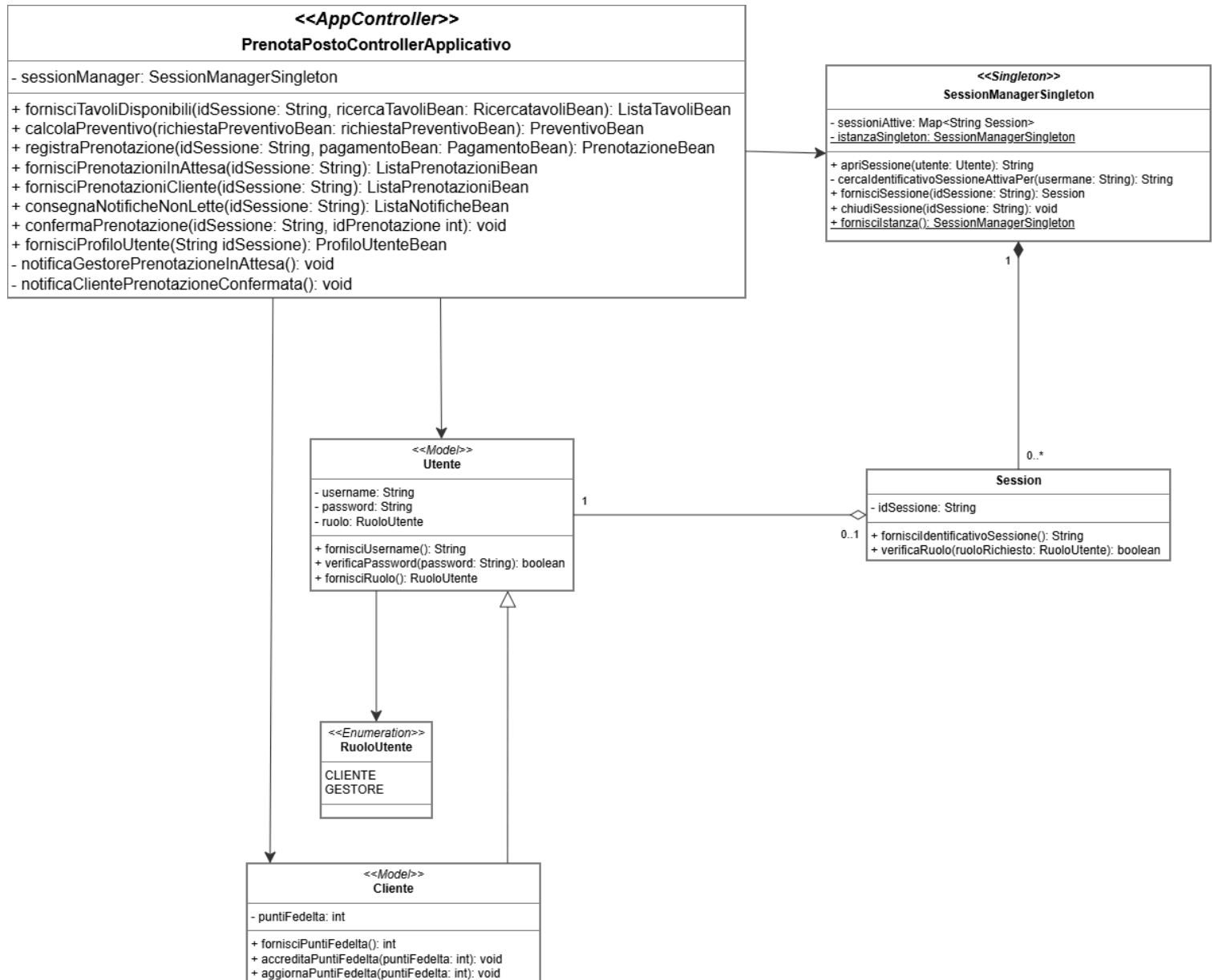
3.1.1 VOPC



3.2 Design Patterns

In questa sezione vengono descritti i design pattern utilizzati per lo sviluppo di **QuestTable**, evidenziando come la loro applicazione contribuisca alla manutenibilità e alla robustezza del sistema.

3.2.1 Singleton Pattern.



Il pattern **Singleton** è stato adottato per gestire la sessione centralizzata delle sessioni applicative.

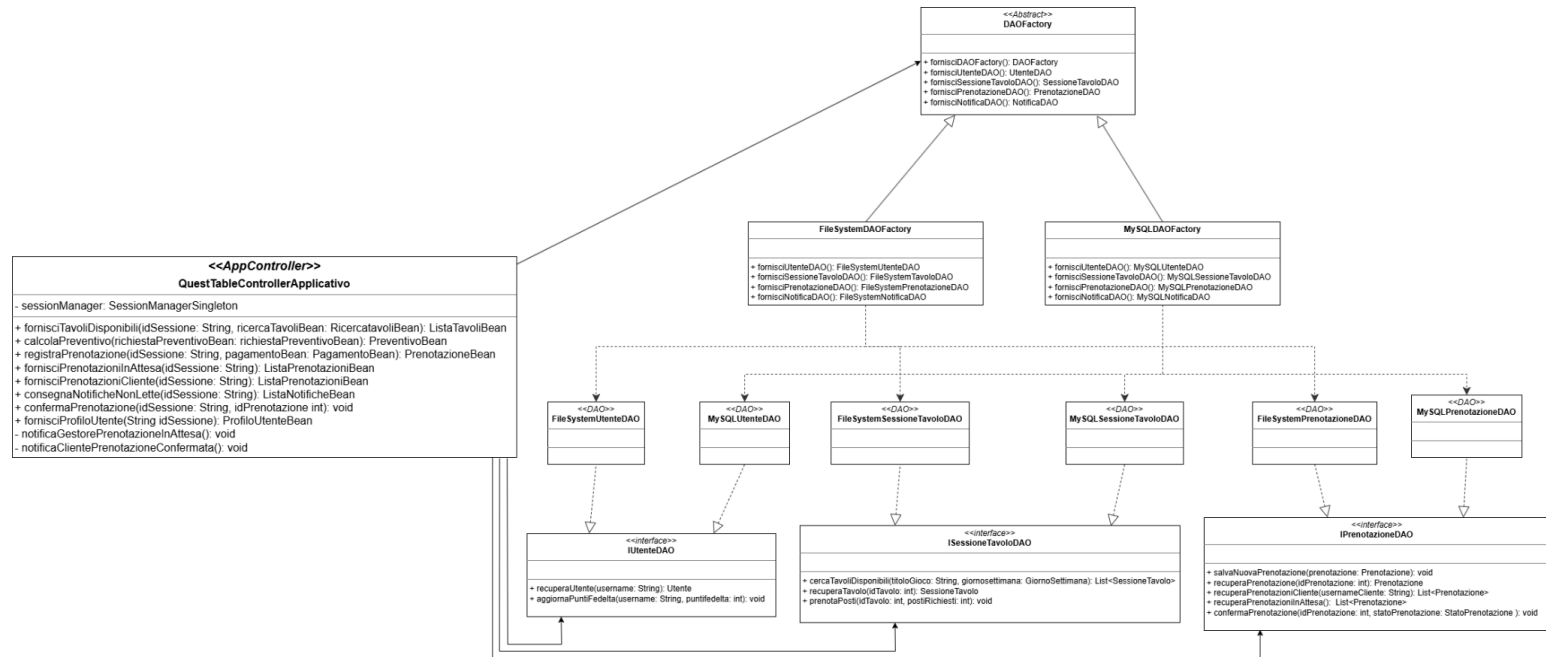
Motivazione: Il sistema deve fornire un unico punto di accesso per aprire, recuperare e chiudere le sessioni degli utenti autenticati. In questo modo si evita che controller diversi gestiscano copie indipendenti dello stato di autenticazione.

Applicazione: La classe **SessionManagerSingleton** implementa questo pattern. Essa mantiene le sessioni attive e permette al controller applicativo di recuperare la sessione associata a uno specifico identificativo. Le singole sessioni non sono Singleton:

possono esistere più sessioni, ciascuna associata a un utente autenticato. È invece unico il manager che coordina tali sessioni.

Vantaggio: La gestione dell'autenticazione e del logout rimane coerente tra le diverse interfacce dell'applicazione, sia JavaFX sia CLI, evitando duplicazioni nella logica di sessione.

3.2.2 Abstract Factory Pattern.



Il pattern **Abstract Factory** è stato adottato per gestire la creazione degli oggetti DAO, separando la logica applicativa dalla tecnologia concreta usata per la persistenza dei dati.

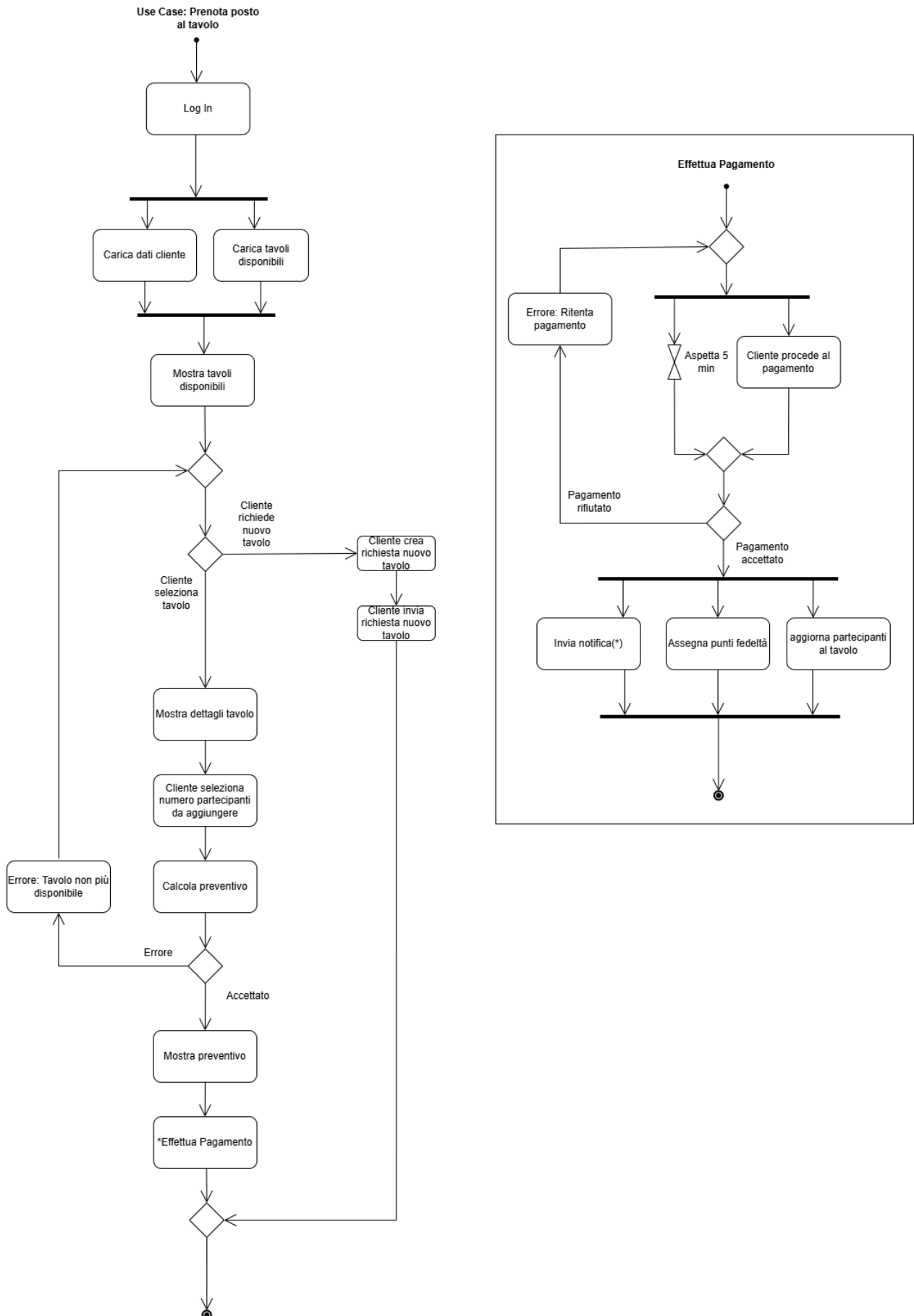
Motivazione: Il sistema QuestTable deve poter funzionare con diverse modalità di persistenza, come demo in memoria, file system e MySQL. Il controller applicativo non deve conoscere quale implementazione concreta viene utilizzata, ma deve lavorare soltanto con le interfacce DAO.

Applicazione: È stata definita la classe astratta DAOFactory, che dichiara i metodi per ottenere i DAO necessari al caso d'uso, ad esempio fornisciUtenteDAO(), fornisciSessioneTavoloDAO() e fornisciPrenotazioneDAO(). Le factory concrete come FileSystemDAOFactory e MySQLDAOFactory, implementano questi metodi restituendo le rispettive implementazioni concrete dei DAO. La scelta della factory da utilizzare avviene in base alla configurazione della persistenza selezionata all'avvio dell'applicazione.

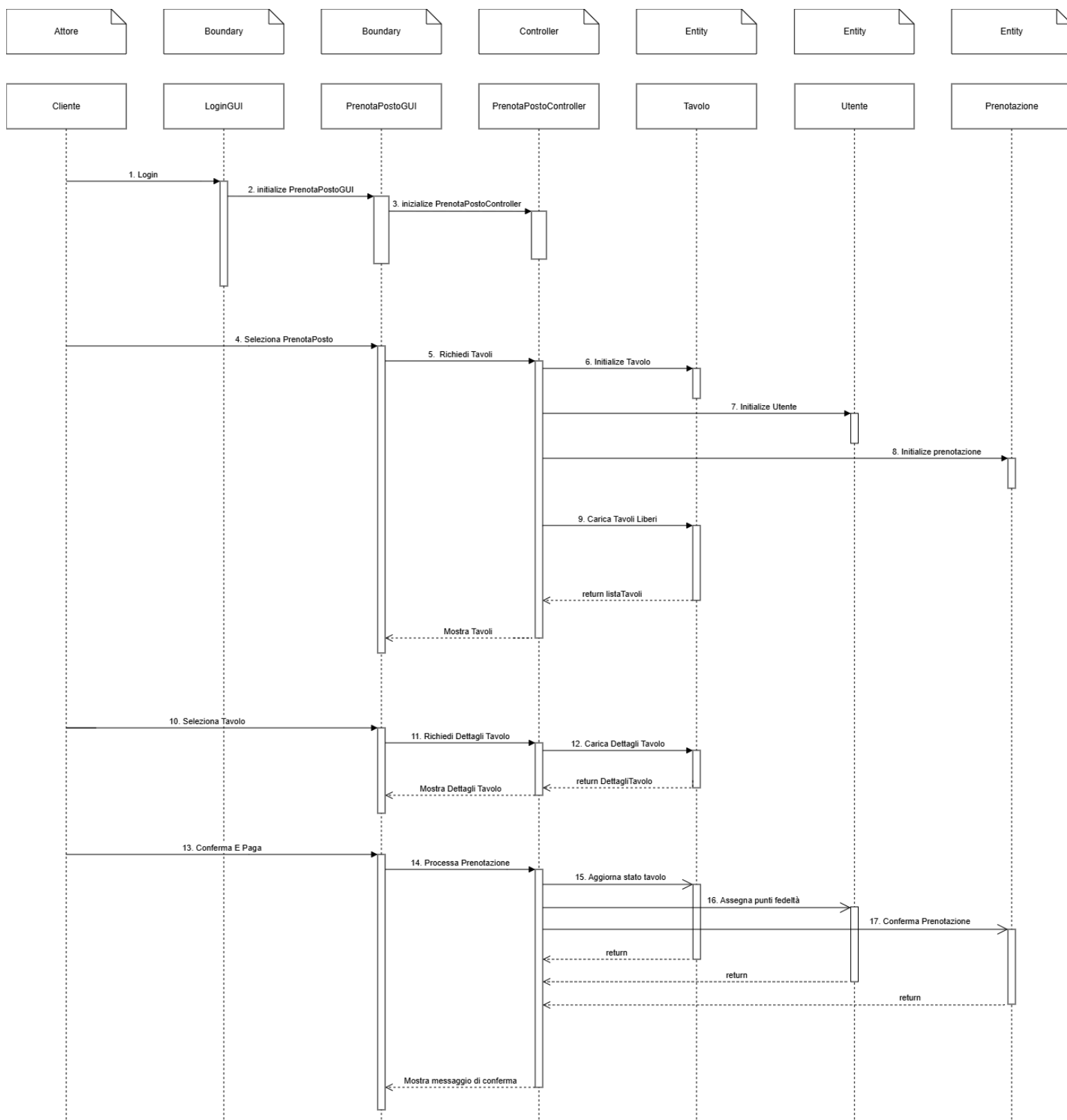
Vantaggio: Questa soluzione rende il sistema più estensibile e manutenibile: per aggiungere una nuova modalità di persistenza non è necessario modificare la logica del controller applicativo, ma è sufficiente introdurre una nuova factory concreta e nuove implementazioni DAO. In questo modo viene rispettato il principio di separazione delle responsabilità e il controller resta indipendente dai dettagli tecnici della persistenza.

3.3 Activity Diagram UC: Prenota posto al tavolo.

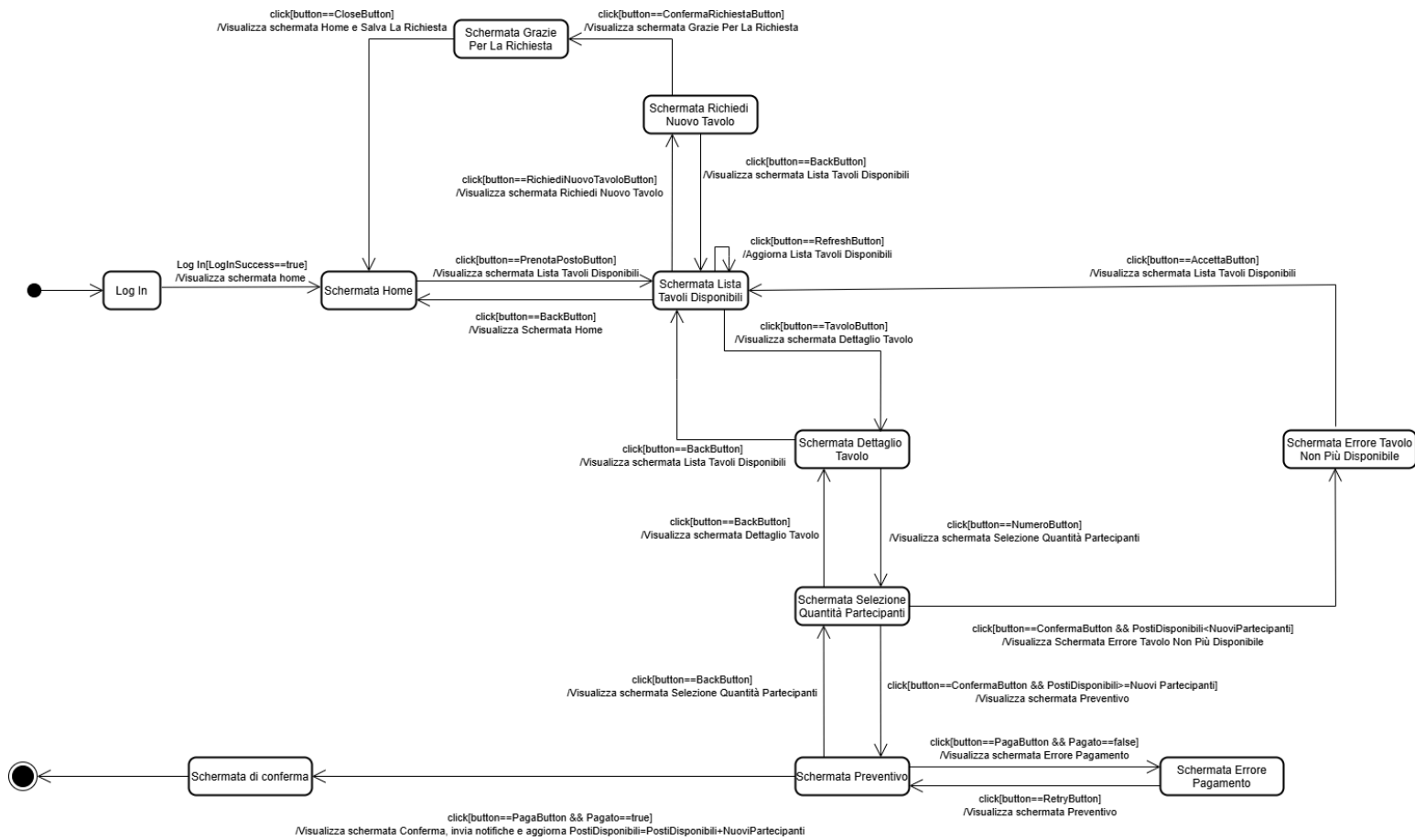
Invia notifica(*): Il sistema invia una notifica al gestore, confermando che la prenotazione e transazione sono state effettuate con successo.



3.4 Sequence Diagram UC: Prenota posto al tavolo.



3.5 State Diagram UC: Prenota posto al tavolo.



4. TESTING

4.1 BeanValidationTest: verifica la corretta validazione dei bean usati nello scambio dati tra controller grafici e controller applicativo.

4.2 PrenotaPostoControllerApplicativoTest: verifica il comportamento del controller applicativo responsabile del caso d'uso "Prenota posto al tavolo". In particolare, controlla la ricerca dei tavoli disponibili, il calcolo del preventivo, la registrazione della prenotazione dopo il pagamento, il recupero delle prenotazioni e la conferma da parte del gestore.

4.3 ModelBehaviorTest: verifica il comportamento delle entity del model, ad esempio il calcolo dell'importo, la gestione dei posti disponibili, lo stato della prenotazione e l'accredito dei punti fedeltà.

5. GITHUB REPOSITORY

<https://github.com/Matteo-Marcoccia/QuestTable.git>

6. SONARCLOUD

https://sonarcloud.io/project/overview?id=Matteo-Marcoccia_QuestTable

7. VIDEO

https://drive.google.com/file/d/1PbtC6uSjgA_gsXszAVV6qhZPbZQatzUn/view?usp=sharing